

04 — Judge System

Objetivo. Documentar cómo Puglit evalúa los artefactos que sus agentes producen: el panel de jurados (*Grand Jury*) que puntúa los blueprints de la competencia genética, el *blend* determinístico que corrige el ruido del LLM, la revisión adversarial pre-entrega de tres lentes, y los mecanismos de votación, desempate, confianza, mitigación de sesgo y auditabilidad. Todo verificado contra el código real en `web/lib/tournament.ts`, `web/lib/adversarial-review.ts`, `web/lib/playbooks.ts` y `web/lib/swarm-fitness.ts`.

1. Why a Judge Exists

Puglit corre **tres equipos genéticos** (`Lean`, `Enterprise`, `Hacker`) que divergen sobre la misma idea, cada uno con su propia filosofía **y su propio modelo** (Qwen, DeepSeek-Coder, Devstral). El problema es directo: tres blueprints distintos compiten, y alguien tiene que elegir cuál pasa a construirse en la iteración 3. Esa elección no puede ser arbitraria ni quedar en manos de un solo modelo.

Un LLM-as-judge resuelve el problema de selección, pero introduce dos vicios conocidos:

1. **Ruido.** Un solo juez es inconsistente: la misma comparación puede dar veredictos distintos según el orden, el fraseo o la suerte del muestreo.
2. **Sesgo de complacencia.** Los modelos tienden a ser *cheerleaders*: puntúan alto, no discriminan, y empatan para no comprometerse.

Puglit ataca ambos de frente. El juez no es un oráculo único sino un **panel** cuya señal subjetiva se **mezcla con una métrica objetiva y determinística**, con una postura explícitamente **adversarial** codificada en el prompt y en el playbook. Y antes de que cualquier deliverable llegue al juez, pasa por una **revisión adversarial de tres lentes** que lee el código real (no el claim) y emite un veredicto `SHIP / SHIP-WITH-CAVEATS / BLOCK`.

La filosofía del sistema en una frase, tomada del playbook:

"A confident answer is not a correct one." — `PLAYBOOK.adversarial`, `web/lib/playbooks.ts`

El juez existe porque la selección genética sin un evaluador discriminante, calibrado y auditable colapsa: todos los equipos "ganan" y la presión evolutiva desaparece.

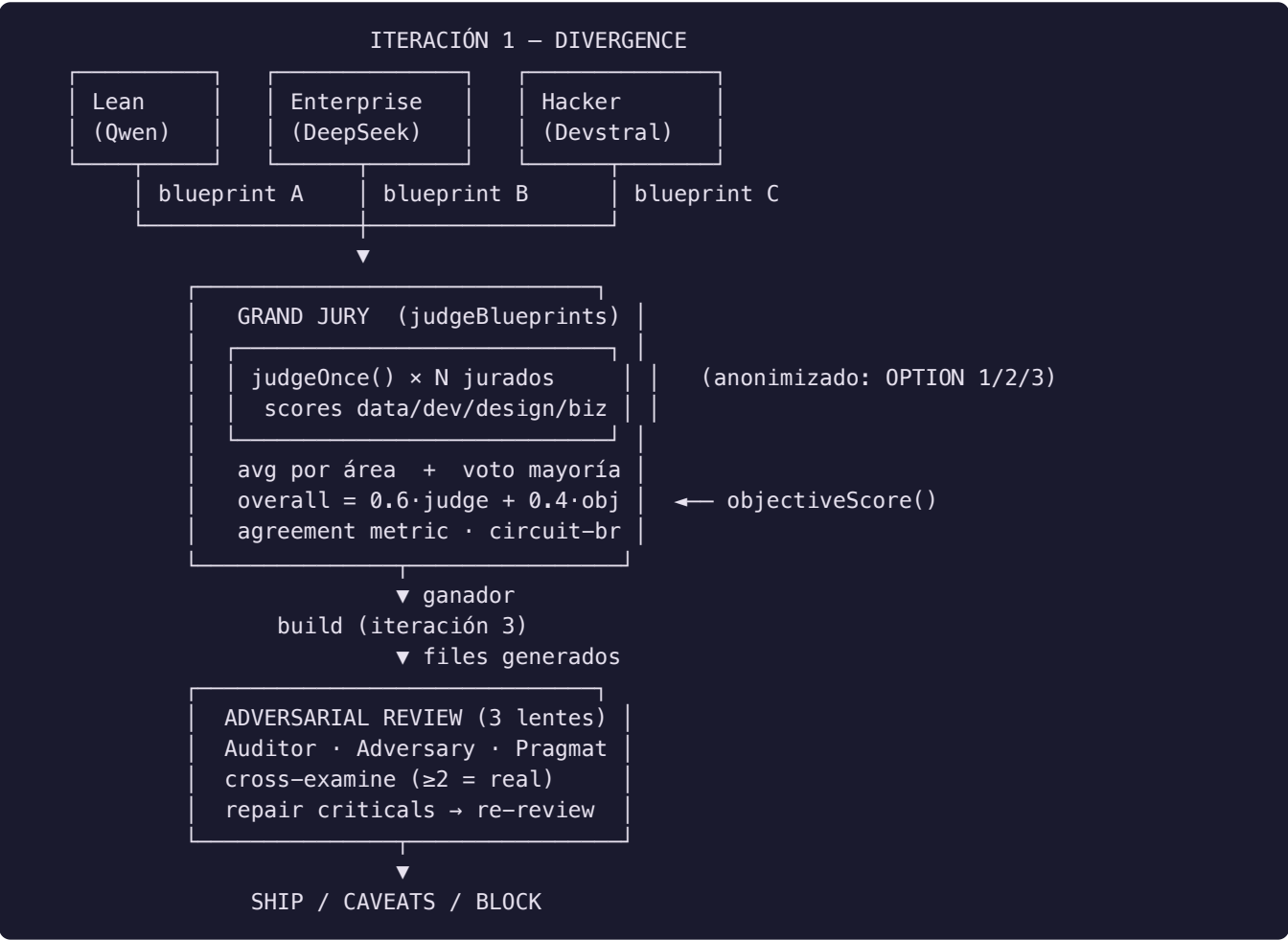
2. Judge Architecture

Hay **dos sistemas de evaluación distintos**, en dos momentos del pipeline:

Sistema	Archivo	Momento	Qué evalúa	Salida
Grand Jury (tournament judge)	<code>web/lib/tournament.ts</code>	Iteración 1, después de la divergencia	Los 3 blueprints rivales	Score por área + ganador
Adversarial Review (pre-delivery)	<code>web/lib/adversarial-review.ts</code>	Antes de entregar el deliverable final	El código generado del ganador	<code>SHIP / SHIP-WITH-CAVEATS / BLOCK</code>

Son ortogonales: el primero **elige** (selección competitiva), el segundo **inspecciona** (control de calidad). Ambos comparten la misma postura adversarial y el mismo principio de no confiar en una sola opinión.

Flujo de extremo a extremo



El orquestador que dispara todo esto es `runDivergence()` en `tournament.ts`, invocado desde `web/app/api/genetic/tournament/route.ts`. La revisión adversarial se invoca dentro de `buildAdvance` en `web/lib/app-builder.ts:1537`.

3. Judge Models (Modelos · Pesos)

El jurado real

El panel se configura por entorno. Por defecto es **un único juez**, `MODELS.premium`; con `PUGLIT_JURY_MODELS` se vuelve un **triunvirato** multi-modelo. Del código (`tournament.ts:82` y `:122`):

```
const JURY_MODELS = (process.env.PUGLIT_JURY_MODELS || "")
  .split(",")
  .map(s => s.trim())
  .filter(Boolean)
// ...
const jury = JURY_MODELS.length ? JURY_MODELS : [MODELS.premium]
```

Configuración	Valor	Jurados	Comentario
Default	(vacío)	1 (<code>MODELS.premium</code>)	Un solo juez premium
Panel	<code>PUGLIT_JURY_MODELS="gpt-oss:20b,qwen2.5-coder:32b,deepseek-r1:32b"</code>	N (coma-separados)	Triunvirato; ejemplo del docstring del código

Nota de infraestructura. En la A40 de una sola GPU, los jurados corren **secuencialmente** (`for (const m of jury)`), porque el modelo se *swapea* en VRAM. No hay paralelismo de panel en una sola tarjeta — es un costo de latencia consciente.

El blend 60/40 con `objectiveScore`

La crítica central que el sistema se hace a sí mismo es "*el juez es ruidoso*". La respuesta es no confiar el `overall` puramente al veredicto subjetivo. Por cada equipo, el score final se calcula así (`tournament.ts:149-153`):

```
const judgeOverall = Math.round((data + dev + design + business) / 4) // promedio de las 4 áreas
const obj = objectiveScore(d.blueprint) // 0-100 determinístico, s
overall = Math.round(0.6 * judgeOverall + 0.4 * obj) // 60% juez / 40% objetivo
```

Componente	Peso	Fuente	Naturaleza
<code>judgeOverall</code>	60%	Panel LLM (promedio de áreas, promediado entre jurados)	Subjetivo, calibrado por rúbrica
<code>objectiveScore(blueprint)</code>	40%	<code>web/lib/swarm-fitness.ts</code> , pura estructura	Determinístico, reproducible

`objectiveScore()` no toca ningún LLM. Parte de una base de 40 y suma/resta por hechos medibles del blueprint:

- +8 si hay ≥ 1 tabla, +6 si hay ≥ 3 tablas
- **+14** si *todas* las tablas tienen ≥ 2 columnas (schemas reales, no stubs)
- **+14** si `routes  $\geq$  max(2, #tablas)` (las operaciones cubren las entidades)
- +10 si hay ≥ 2 páginas, +4 si hay ≥ 3
- **Penalización** de hasta -20: rutas que referencian tablas que el blueprint nunca declara (riesgo de *phantom-table*, -4 por cada una)

El resultado se clampea a `[0, 100]` . Este 40% es lo que ancla la selección a la tierra: un blueprint que el juez adula pero que tiene rutas fantasma o tablas-stub pierde puntos que el LLM no le iba a quitar.

4. Scoring Categories

El juez **no** usa las categorías de marketing habituales (Architecture, Security, UX, Maintainability, Innovation, Performance). Usa **cuatro disciplinas reales**, definidas en el schema `JUDGE_SCHEMA` y en el system prompt de `judgeOnce` (`tournament.ts:34-49` , `:98-102`). El mapeo honesto entre la taxonomía conceptual y lo que el código mide:

Categoría conceptual	Área real en Puglit	Qué evalúa (texto literal del prompt)
Architecture	<code>data</code>	El modelo de datos: tablas/relaciones correctas para este producto, sin entidades core faltantes, sin tablas contaminantes
UX	<code>design</code>	Las páginas/UX: las pantallas que un usuario necesita para usar el producto de punta a punta
Maintainability	<code>dev</code>	La API/operaciones: cada <i>user journey</i> tiene sus rutas (create + read + la acción core del producto), sin <i>dead ends</i>
Innovation + Business model	<code>business</code>	Fidelidad al producto + completitud de features + un modelo de plata coherente (sin pricing si es gratis, sin signup si es público)
Security	<i>transversal</i> (penalización)	No es un eje numérico separado: se penaliza dentro de las 4 áreas (SQL por concatenación, secrets en código) y se ataca en serio en la revisión adversarial (§8)
Performance	<i>no medido por el juez</i>	El juez evalúa diseño, no runtime; el comportamiento en ejecución lo cubre el <i>runtime gate</i> (<code>build-local.mjs</code>), fuera del scope de este doc

Cada disciplina se puntúa **0-100** con una **rúbrica de anclas explícitas**, no "vibes" (`tournament.ts:103`):

Rango	Significado
90-100	Ships as-is
70-89	Minor fixes
50-69	Falta una pieza core
<50	Roto / incoherente

El prompt penaliza explícitamente: tablas alucinadas/no usadas, rutas *dead-end*, ausencia de la acción core del producto, patrones inseguros. Y ordena: *"Be critical and DISCRIMINATING; do not tie."*

El `overall` por área que se persiste en `byTeam` es el promedio simple de las cuatro (`Math.round((data+dev+design+business)/4)`) **antes** del blend 60/40 descrito en §3.

5. Voting Logic

Cada jurado hace dos cosas independientes en `judgeOnce()` :

1. **Puntúa** las 4 áreas de cada candidato (más una crítica de una frase).
2. **Vota** un ganador (`winner: <option number>`).

El panel (`judgeBlueprints`) agrega así:

Scores por área → **promedio entre jurados**. Para cada equipo, se promedian los scores de cada área a través de todos los jurados que lo evaluaron (`tournament.ts:145`):

```
const avg = (k) => Math.round(ss.reduce((s, x) => s + x[k], 0) / ss.length)
```

Las críticas de cada jurado se deduplican y se quedan las **3 primeras**, unidas por `·`, para que el diario y la UI vean la visión completa del panel.

Ganador → **voto de mayoría**. Se cuentan los votos de `winner` de cada jurado y gana el más votado (`tournament.ts:157-160`):

```
const votes = {}
for (const v of verdicts) if (v.winner) votes[v.winner] = (votes[v.winner] || 0) + 1
let winner = Object.entries(votes).sort((a, b) => b[1] - a[1])[0]?.[0]
```

Es importante notar la **separación de mecanismos**: el *score por área* (y el `overall` con blend) se promedia; el *ganador* se decide por voto de mayoría, no por el score agregado más alto. El score agregado solo entra como **desempate** (§6). Esto evita que un jurado outlier que infló un score arrastre la elección si la mayoría votó otra cosa.

Cuando el panel es de un solo jurado (default), "mayoría" es trivialmente su voto, y `judgeOnce` ya tiene su propio fallback interno: si el modelo no devuelve un `winner` válido, elige el de mayor `overall` (`tournament.ts:114-115`).

6. Tie Breaking

El desempate está codificado explícitamente (el código lo llama "*el desempate*", `tournament.ts:156-163`):

```
const ranked = Object.entries(byTeam).sort((a, b) => b[1].overall - a[1].overall)
let winner = Object.entries(votes).sort((a, b) => b[1] - a[1])[0]?.[0]
const topVotes = winner ? votes[winner] : 0
const tied = Object.values(votes).filter(n => n === topVotes).length > 1
if (!winner || tied) winner = ranked[0]?.[0] || designs[0]?.team
```

La cadena de resolución, en orden:

Caso	Resolución
Voto de mayoría claro	El equipo más votado gana
Empate en votos (≥ 2 equipos con el mismo <code>topVotes</code>)	Gana el de mayor overall agregado (con el blend 60/40 ya aplicado)
Sin votos (ningún jurado emitió <code>winner</code>)	Gana el de mayor <code>overall</code> agregado
Todo lo anterior falla	<code>designs[0]?.team</code> (primer equipo, garantía de no-null)

El `overall` agregado como criterio de desempate es deliberado: incorpora el `objectiveScore` determinístico, así que el desempate **no** es subjetivo — se rompe con la métrica estructural medible, no con otra opinión del LLM.

7. Confidence Scores (Inter-Judge Agreement)

El sistema responde a la pregunta "*¿le creemos al juez?*" con una métrica de **acuerdo inter-jurado**: la fracción de jurados que votaron por el ganador eventual de la mayoría (`tournament.ts:134-139`):

```
const winnerVotes = {}
for (const v of verdicts) if (v.winner) winnerVotes[v.winner] = (winnerVotes[v.winner] || 0) + 1
const topAgree = Math.max(0, ...Object.values(winnerVotes))
recordMetric("judge_agreement", verdicts.length > 1 ? topAgree / verdicts.length : 1, { jurors: ... })
```

Acuerdo	Interpretación
<code>1.0</code>	Unanimidad — o panel de 1 jurado (por convención, confianza máxima)
<code>0.66</code>	2 de 3 votaron al ganador — señal razonable
<code>0.5</code>	Panel empatado — la señal de <i>fitness</i> es ruidosa

Esta métrica **no altera la selección** (no cambia quién gana); se **registra** vía `recordMetric` en la tabla `puglit_metrics` para el scorecard. Es un termómetro: un acuerdo bajo sostenido significa que las comparaciones son demasiado parejas o que el juez es inconsistente, y que el 40% objetivo está cargando más peso del esperado.

`judge_agreement` es uno de los nombres canónicos rastreados en `web/lib/swarm-metrics.ts`, lo que permite agregarlo a lo largo de muchos builds.

8. Bias Mitigation

Puglit aplica mitigación de sesgo en dos frentes: **anonimización** y **postura adversarial**.

Anonimización (Option 1/2/3)

Los candidatos nunca se presentan al juez con su identidad de equipo. Se serializan como tarjetas neutras numeradas — `OPTION 1`, `OPTION 2`, `OPTION 3` — sin nombre de equipo ni de modelo

(tournament.ts:87-88):

```
OPTION 1
kind: ...
summary: ...
tables (N): ...
routes (N): ...
pages (N): ...
```

El prompt refuerza la regla: "Judge only on merit, ignore house style" y "Judge the WHOLE design of each candidate (never mix pieces across candidates)" (tournament.ts:103). El juez no sabe que OPTION 2 es el equipo Hacker corriendo Devstral. Esto elimina el sesgo de favorecer a un modelo o filosofía por reputación en lugar de por el mérito del blueprint concreto.

El docstring de judgeOnce lo nombra textualmente: "ONE judge scores each design ANONYMIZED (Option 1/2/3)".

Postura adversarial

El system prompt del jurado empieza con: "Judge like an ADVERSARIAL reviewer, not a cheerleader" e inyecta PLAYBOOK.adversarial completo (tournament.ts:92-96). El playbook (playbooks.ts:58-68) codifica las tres lentes ortogonales con instrucción explícita de sesgo a rechazar en la lente de seguridad ("Bias to REJECT"). Esto contrarresta el sesgo de complacencia de los LLMs.

El segundo control: revisión adversarial de tres lentes

adversarial-review.ts implementa el patrón addyosmani/adverse de forma nativa. Un solo modelo, tres personas — más barato que multi-modelo, y el consenso entre lentes corta el sesgo de anclaje del modelo único:

Lente	Foco (carril estricto)	Sesgo
Auditor	Lógica y corrección: math de dominio, edge cases, off-by-one, mismatches de naming/schema, valores hardcoded	neutral
Adversary	Abuso y seguridad: SQLi, auth faltante / IDOR, race conditions, double-booking/double-spend, input sin validar, secrets	REJECT
Pragmatist	Salud de diseño: lógica de negocio en routes en vez de lib/ , violaciones DRY, rutas muertas, tests faltantes, over-engineering	neutral

Cada lente se queda en su carril ("STAY IN YOUR LANE") para no duplicar hallazgos artificialmente. Después viene la cross-examination determinística: los hallazgos se agrupan por una clave normalizada (key0f = archivo + título normalizado), y se clasifican (adversarial-review.ts:103-117):

```
hallazgo reportado por ≥2 lentes → crossValidated (señal real)
hallazgo reportado por 1 lente   → solo           (una corazonada)
```

Solo los **CRITICAL cross-validados** disparan una pasada de reparación acotada (`repairFindings` , máximo 3 archivos), tras la cual se re-revisa desde cero. El veredicto final (`adversarial-review.ts:142-147`):

Condición	Veredicto
Queda un CRITICAL cross-validado o ≥ 2 lentes rechazaron	BLOCK
Hay hallazgos cross-validados, o un CRITICAL solo, o alguna lente no aprobó	SHIP-WITH-CAVEATS
Limpio en todo	SHIP

En `PUGLIT_TRAINING_MODE` (batch de pre-carga del cerebro) se revisa **una sola vez** y se salta la reparación + re-revisión costosa, para no inflar el costo del batch de 100 builds.

9. Auditability

Todo el proceso de juicio deja rastro persistente y reproducible.

Rondas (`puglit_rounds`). Por cada equipo y cada iteración se inserta una fila con score, flag de ganador, las críticas como `notes` , y un snapshot JSONB completo en `artifacts` (`tournament.ts:185-189` , schema en `web/sql/genetic.sql:67-79`):

```
puglit_rounds(id, job_id, iteration, team, role, score, winner, notes, artifacts, created_at)
```

El JSONB `artifacts` guarda el blueprint completo, las métricas, el modelo usado (incluyendo si fue *fallback*) y los scores por área. Esto permite reconstruir, build a build, **por qué** ganó un equipo: qué diseño presentó, qué modelo lo generó, y cómo lo puntuó cada área.

Métricas (`puglit_metrics` vía `recordMetric`). Señales agregables a lo largo de la historia:

Métrica	Cuándo	Qué dice
<code>judge_agreement</code>	cada juicio con panel	acuerdo inter-jurado (§7)
<code>judge_failed</code>	jurado caído	se activó el circuit breaker
<code>adverse_verdict</code>	cada revisión adversarial	<code>1</code> SHIP / <code>0.5</code> CAVEATS / <code>0</code> BLOCK + #confirmados/#solo/#reparados

Logs estructurados. La revisión adversarial loguea su veredicto y los primeros 6 hallazgos con severidad, lente y archivo (`app-builder.ts:1539-1540`), y la degradación del council se loguea cuando un modelo de equipo falla y cae al fallback (`tournament.ts:68`).

Reproducibilidad. El 40% del score (`objectiveScore`) es **determinístico**: dado el mismo blueprint, da el mismo número, siempre, sin LLM. Eso significa que cualquier auditoría puede recomputar la mitad objetiva del veredicto offline y verificar que la selección no fue puro capricho del modelo.

Circuit breaker (graceful degradation)

Si **todo** el jurado falla (todos los modelos caídos o devolviendo basura no parseable), el pipeline **no se bloquea** (`tournament.ts:127-133`):

```
if (!verdicts.length) {
  recordMetric("judge_failed", 1)
  // degrada a "draft mode": gana el blueprint MÁS COMPLETO (rutas + tablas)
  const winner = designs.sort((a, b) =>
    (b.routes.length + b.tables.length) - (a.routes.length + a.tables.length))[0]?.team
  // todos los equipos reciben overall=60, crítica "draft mode (jurado no disponible)"
}
```

El fallback elige por la heurística estructural más simple (más rutas + tablas), marca el evento en métricas, y deja una traza explícita ("*draft mode (jurado no disponible)*") en las críticas, de modo que una auditoría posterior sabe que ese build no pasó por el juez completo.

10. Benchmark Results

Por honestidad intelectual: lo que está **medido en infraestructura** hoy y lo que está **diseñado pero no aún cuantificado en un reporte público**.

Lo que está instrumentado (se mide en cada build)

Señal	Mecanismo	Estado
<code>judge_agreement</code>	<code>recordMetric</code> , panel multi-jurado	Instrumentado; se acumula en <code>puglit_metrics</code>
<code>adverse_verdict</code>	<code>recordMetric</code> tras la revisión adversarial	Instrumentado (1 / 0.5 / 0)
<code>judge_failed</code>	circuit breaker	Instrumentado
<code>build_success</code>	<code>swarm-checks</code> (high issues = 0)	Instrumentado
<code>objectiveScore</code>	determinístico por blueprint	Reproducible offline

Estas métricas se persisten por build, pero **el agregado estadístico (medias, distribuciones, calibración del juez vs. el gate de runtime) todavía no está consolidado en un reporte publicable**. Lo que existe es la *instrumentación*, no aún la *serie temporal analizada*.

La evidencia cualitativa real

El punto de validación duro documentado hasta hoy no viene del juez de blueprints sino del **runtime gate**, que es el verdadero árbitro de verdad:

- **Stayforge** (litmus test, clon de Airbnb): 80 archivos, 8 tablas, anti-double-booking con `EXCLUDE USING gist` , pricing en centavos-enteros determinístico, refund por snapshot de política, reviews doble-ciego. **El runtime gate expuso 7 bugs reales** que un compilado en verde no veía. Esto valida

la tesis del sistema completo: el juicio subjetivo y el compilado limpio **no bastan** — por eso el juez es solo una etapa y la verdad final la da bootear la app.

- **Batch de 100 builds** de pre-carga del cerebro (`infra/train-brain.sh`): en curso. Genera el corpus de diary/exemplars/métricas que eventualmente permitirá reportar acuerdo inter-jurado medio, tasa de `BLOCK` / `CAVEATS` y correlación juez↔runtime con números.

Limitaciones declaradas (honestidad)

1. **El juez es ruidoso por construcción** — el sistema lo admite y lo compensa con el 40% objetivo, pero no lo elimina.
2. **En 1 GPU el panel es secuencial** — un triunvirato triplica la latencia del juicio; el default de 1 jurado es una decisión de costo, no de calidad.
3. **El acuerdo de un panel de 1 jurado es 1.0 por convención**, no por consenso real — la métrica solo es informativa con `PUGLIT_JURY_MODELS` configurado.
4. **No hay aún calibración publicada** del juez de blueprints contra el resultado del runtime gate; medir si "el ganador del juez" correlaciona con "el que pasa el gate sin bugs" es trabajo pendiente que el batch de 100 builds habilita.

Archivos fuente verificados: `web/lib/tournament.ts`, `web/lib/adversarial-review.ts`,
`web/lib/playbooks.ts`, `web/lib/swarm-fitness.ts`, `web/lib/swarm-metrics.ts`, `web/lib/app-builder.ts`, `web/sql/genetic.sql`.